Password Reset Security Checklist

Educational Bug Bounty Series • Authorized Security Research Only

■ Password Reset & Account Recovery — Security Checklist

Contributed by Cyber_Spy.404 | Educational Bug Bounty Series

Scope: *Use controlled accounts and inboxes. Never attempt to reset another person's password or reuse their tokens.*

■ Executive Summary

Password-reset flows are sensitive authentication mechanisms. The supplied two-page checklist describes risks involving token validation, Host header injection, referrer leakage, HTML injection in email, company-invitation flows, token removal or mutation, null/old/oversized tokens, request-method and content-type variations, response manipulation, cross-domain token use, and race conditions.

■ Core Testing Methodology

1. Use a dedicated test account and controlled email inbox to request a reset.
2. Verify reset tokens are cryptographically random, sufficiently long, single-use, and short-lived.
3. Confirm tokens are bound to the intended account and purpose.
4. Ensure generated links use a configured canonical domain, not untrusted Host or forwarding headers.
5. Check that reset tokens are not leaked through referrer headers, analytics, logs, or response bodies.
6. Verify malformed, missing, expired, reused, null-like, or altered tokens fail safely.
7. Check that HTTP method and content-type changes do not bypass server-side validation.
8. Ensure responses do not reveal whether an account exists.
9. Verify reset requests are rate-limited and monitored.
10. Review concurrent reset attempts and token invalidation behavior using the test account.
11. Review invitation and company-email flows separately; do not add unexpected password fields to real users' requests.
12. Document the expected behavior and retest remediation.

■ High-Value Bug Bounty Tips & Edge Cases

- **Token binding:** A token generated for one account must not work for another account or domain.
- **Host header:** Reset links should use a configured canonical origin.
- **Referrer leakage:** Sensitive tokens in URLs can leak to third-party resources; apply a restrictive Referrer-Policy and avoid loading third-party content on reset pages.
- **Token mutations:** Empty, altered, expired, reused, or malformed tokens should be rejected consistently.
- **Old token arrays:** Previous tokens must not remain valid after successful reset unless explicitly designed and safely constrained.
- **Response manipulation:** Client-side changes to a response must not change server-side verification state.
- **Cross-domain use:** The source notes multi-domain applications; tokens should be bound to the intended application and reset purpose.
- **Race conditions:** Concurrent requests should not allow a token to be reused or a password reset to be applied unpredictably.
- **SQL injection and parser variations:** Treat these as code-review/testing categories; do not use destructive payloads on production.

- **Source limitation:** The checklist references an external account-takeover article and contains several terse examples. This rewrite summarizes categories without treating each as a verified bypass.

■■ Educational Checklists & Safe Examples

```
Target: [Your-Domain]
Test account: [Controlled-Test-Account]
Token expiry enforced: [Yes/No]
Token single-use: [Yes/No]
Token bound to account/domain: [Yes/No]
Canonical reset URL: [Yes/No]
Token exposed in referrer/logs: [Yes/No]
Rate limiting: [Yes/No]
Evidence: [Redacted]
```

Example of a safe reset request shape:

```
POST /account/password-reset HTTP/1.1
Host: [Configured-Canonical-Domain]
Content-Type: application/json

{
  "email": "[CONTROLLED-TEST-EMAIL]",
  "token": "[REDACTED-RESET-TOKEN]",
  "new_password": "[TEST-ONLY-PASSWORD]"
}
```

- [ ] Tokens are unpredictable, expiring, single-use, and account-bound.
- [ ] Reset URLs use a configured origin.
- [ ] Responses avoid account enumeration.
- [ ] Tokens are not logged or sent to third parties.
- [ ] Rate limits and audit logging are in place.

■■ Mitigation & Remediation

Use high-entropy single-use tokens with short expiry; bind them to account and purpose; invalidate them after use or credential change; construct links from a trusted canonical origin; prevent token leakage through referrers and logs; rate-limit requests; use generic responses; and test concurrency and cross-domain boundaries.

■ Source Notes

Based on the supplied two-page *reset pass checklist.pdf*. Page 1 lists token use, Host header injection, HTML injection, referrer leakage, company-email invitation behavior, CRLF, email injection, and token removal. Page 2 lists zero/null and old tokens, SQLi category, method/content-type changes, response manipulation, long tokens, cross-domain token usage, response-body leakage, Unicode/race-condition notes, and email-domain variations.

Educational use only. Use only controlled accounts and inboxes.

Educational use only. Test only systems you own or are explicitly authorized to assess. Confirm redistribution rights for source-derived material.